

11

Building a Calculator Using .NET MAUI Blazor

In this chapter, we will explore a .NET **Blazor** app embedded within a .NET MAUI app. We will write part of the calculator app using Blazor and host that in .NET MAUI using **BlazorWebView**. We will also communicate between Blazor and .NET MAUI.

In this chapter, we are going to cover the following topics:

- What is Blazor?
- Exploring the differences between .NET MAUI projects and .NET MAUI Blazor projects
- Using HTML and CSS to define a UI
- Using XAML controls with HTML in a WebView
- Writing C# code that will integrate with XAML controls and HTML controls
- Working with the main .NET MAUI window to resize it to its contents

Technical requirements

You will need to have Visual Studio for Mac or PC installed, as well as the .NET MAUI components. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment. The source code for this chapter is available in this book's GitHub repository:

<https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition>.

Project overview

In this chapter, you will discover .NET Blazor and how to use it with .NET MAUI to develop a UI for an app. We will explore the different options available for hosting a Blazor app within a .NET MAUI app.

Communication between the two apps is a crucial part of the interoperability and this chapter's project will show you how to send data from .NET MAUI to Blazor and vice versa.

What is Blazor?

.NET Blazor is a web framework built on top of .NET. Blazor applications run in the browser by using **WebAssembly (WASM)**, or on a server using SignalR. Blazor is one part of the whole ASP.NET ecosystem, and it leverages Razor pages for developing the UI. Blazor uses HTML and CSS to ren-

der a rich UI. Blazor uses a component-based UI, where each component is a Razor markup page. Within a Razor page, you can mix and match HTML, CSS, and C# code. There are three deployment models for a Blazor app:

- **Blazor Server:** In a Blazor Server deployment, the application code runs on the server in an ASP.NET Core app, and it communicates with the UI running in a browser via SignalR.
- **Blazor WebAssembly:** For Blazor WebAssembly, the entire app runs in the browser via WASM. It is an open web standard that makes it possible to run .NET code safely in the browser. WASM provides interoperability with JavaScript.
- **Blazor Hybrid:** Blazor Hybrid is a blend of native .NET and web technologies. Blazor Hybrid apps can be hosted in .NET MAUI, **WPF**, and **Windows Forms** applications. Since the hosts are all .NET, the Blazor runtime runs natively within the same .NET process and renders the Razor page web UI into a WebView control.

Now that we have some general knowledge of Blazor, let's look at the app we will build in this chapter!

Creating the calculator app

In this chapter, we are going to build a calculator app. The UI for the calculator is built using Razor pages in Blazor, but the actual mechanics of the calculator reside in the .NET MAUI app.

Setting up the project

This project, like all the rest, is a **File | New | Project...**-style project. This means that we will not be importing any code at all. So, this first section is all about creating the project and setting up the basic project structure.

Creating the new project

The first step is to create a new .NET MAUI project. Follow these steps:

1. Open Visual Studio 2022 and select **Create a new project**:

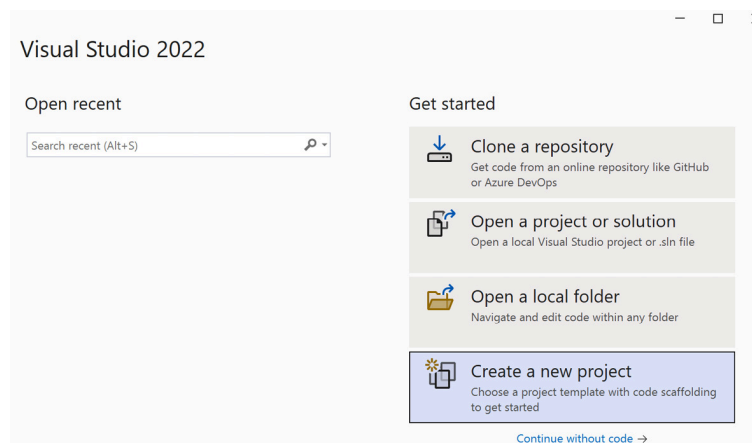


Figure 11.1 – Visual Studio 2022

This will open the **Create a new project** wizard.

2. In the search field, type in **blazor** and select the **.NET MAUI Blazor App** item from the list:

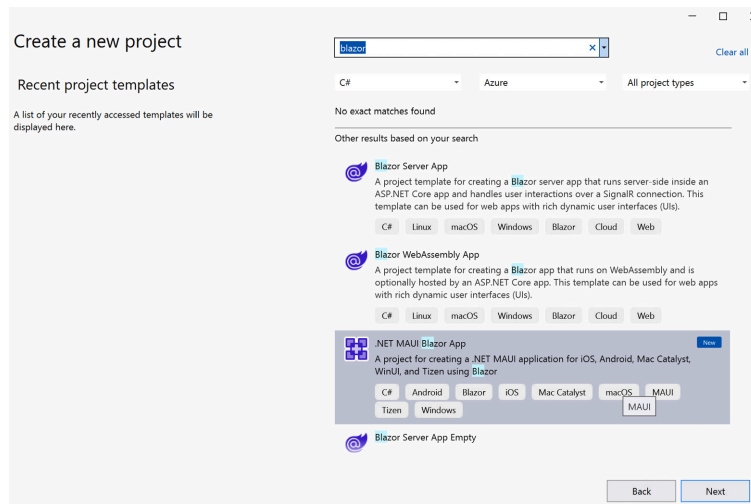


Figure 11.2 – Create a new project

3. Click **Next**.
4. Enter **Calculator** as the name of the app, as shown in the following screenshot:

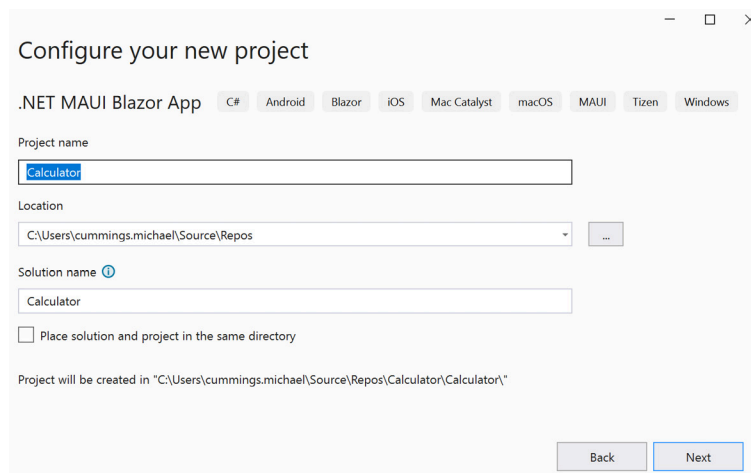


Figure 11.3 – Configure your new project

5. Click **Next**.
- The last step will prompt you for the version of .NET Core to support. At the time of writing, .NET 6 is available as **Long-Term Support (LTS)**, and .NET 7 is available as **Standard Term Support**. For this book, we will assume that you will be using .NET 7:

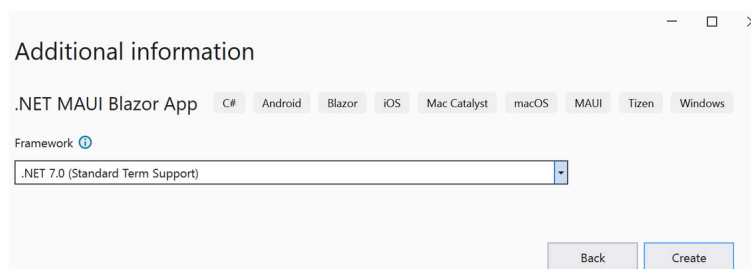


Figure 11.4 – Additional information

6. Finalize the setup by clicking **Create** and wait for Visual Studio to create the project.

That's it for project creation.

Let's continue by reviewing the structure of the app.

Exploring a .NET MAUI Blazor Hybrid project

If you run the project, you will see an app, as shown in *Figure 11.5*. It does not resemble the .NET MAUI app template and has a distinctive web feel to it. Poke around the app a little bit to see how all the visual elements work together. Then, close the application, return to Visual Studio, and continue exploring the project:

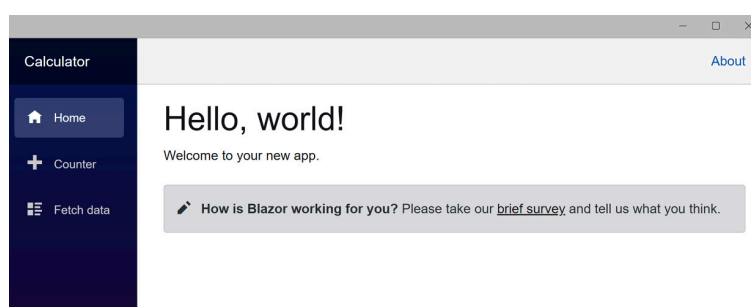


Figure 11.5 – Running the .NET MAUI Blazor template project

The structure of the .NET MAUI Blazor app is a mashup of a .NET MAUI app and a Blazor app. If you look at the **Solution Explorer** window, as shown in *Figure 11.6*, you will see the **Platforms** and **Resources** folders and the **App.xaml**, **MainPage.xaml**, and **MauiProgram.cs** files that are normally present in the .NET MAUI template. The **wwwroot**, **Data**, **Pages**, and **Shared** folders all support the Blazor app. Additionally, you'll find **_Imports.razor** and **Main.razor** at the root of the project:

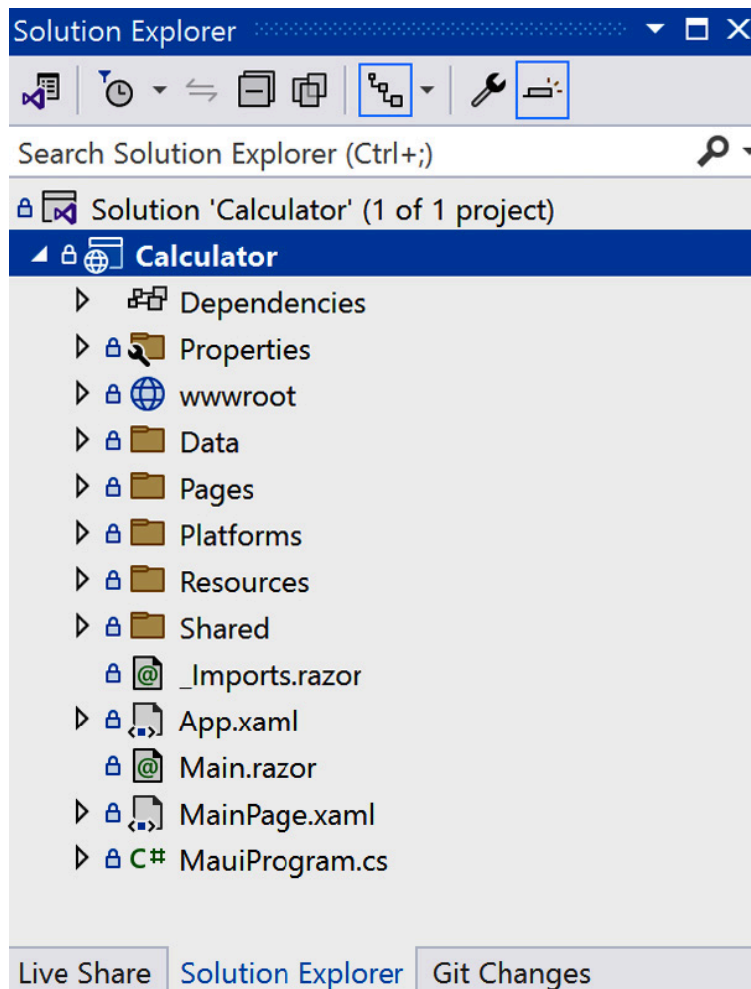


Figure 11.6 – Solution Explorer view of a .NET MAUI Blazor project

If you need a refresher on how a .NET MAUI app is structured and works, refer to *Chapter 1*. Ignoring how a Blazor app works for the moment, let's take a look at how a Blazor app is hosted by .NET MAUI.

Since all .NET MAUI programs start with the **MauiProgram.cs** file, that seems like a good place to start. Open the **MauiProgram.cs** file and inspect its contents. The following snippet highlights the differences for a .NET MAUI Blazor app:

```
builder.Services.AddMauiBlazorWebView();
#if DEBUG
    builder.Services.AddBlazorWebViewDeveloperTools();
    builder.Logging.AddDebug();
#endif
```

The first highlighted line enables the hosting services for the Blazor app, specifically the WebView controller. The second highlighted line enables the developer tools (*F12*) within the WebView control, but only in a debug configuration.

App.xaml and **App.xaml.cs** are essentially the same as in a .NET MAUI template project, but **MainPage.xaml** is not. Open the **MainPage.xaml** file to inspect its contents, as shown here:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage
  xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:Calculator"
  x:Class="Calculator.MainPage"
  BackgroundColor="{DynamicResource PageBackgroundColor}">
  <BlazorWebView x:Name="blazorWebView" HostPage="wwwroot/index.html">
    <BlazorWebView.RootComponents>
      <RootComponent Selector="#app" ComponentType="{x:Type local:Main}" />
    </BlazorWebView.RootComponents>
  </BlazorWebView>
</ContentPage>
```

MainPage has a single control, **BlazorWebView**. This is a wrapper around the native controls to host web pages in apps. The **HostPage** attribute points to the starting page – in this case, **wwwroot/index.html**. The **BlazorWebView.RootComponents** element identifies the starting points for the Blazor application and the location where they are hosted on the page. In this case, **RootComponent Main** is rooted in the element with an ID of **app**.

To see where the **app** element is located, open the **index.html** file in the **wwwroot** folder and inspect its contents, as shown here:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-sc
  <title>Calculator</title>
  <base href="/" />
  <link rel="stylesheet" href="css/bootstrap/bootstrap.min.css" />
  <link href="css/app.css" rel="stylesheet" />
  <link href="Calculator.styles.css" rel="stylesheet" />
</head>
<body>
  <div class="status-bar-safe-area"></div>
  <div id="app">Loading...</div>
  <div id="blazor-error-ui">
    An unhandled error has occurred.
    <a href="" class="reload">Reload</a>
    <a class="dismiss">✕</a>
  </div>
  <script src="_framework/blazor.webview.js" autostart="false"></script>
</body>
</html>
```

The key parts of the file are highlighted in the preceding code. First, the **div** element with **id** set to **app** is where the **Main** component is rooted, or loaded. In the header of the page, the stylesheets are identified. The first stylesheet, **app.css**, is located in the **wwwroot/css** folder of the project. The second stylesheet, **Calculator.styles.css**, is created during the build process from isolated CSS files. **Isolated CSS** is a feature of ASP.NET that allows you to create your styles close to the elements that use them. They are isolated from other components, so they won't inadvertently

bleed into other parts of your application. Finally, the `_framework/blazor.webview.js` file is imported and is responsible for doing all the heavy lifting of getting your Blazor components rendered at the right location on the page.

The final piece for us to review before we move on to creating the rest of the application is the Blazor components. **Main.razor** is a routing file that directs the Blazor runtime to the starting component, **MainLayout.razor**, as shown in the following code:

```
<Router AppAssembly="@typeof(Main).Assembly">
  <Found Context="routeData">
    <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
    <FocusOnNavigate RouteData="@routeData" Selector="h1" />
  </Found>
  <NotFound>
    <LayoutView Layout="@typeof(MainLayout)">
      <p role="alert">Sorry, there's nothing at this address.</p>
    </LayoutView>
  </NotFound>
</Router>
```

The **MainLayout.razor** file defines the basic layout of the page, with a navigation bar on the left, and the main body consuming the remainder of the page, as shown in the following code:

```
@inherits LayoutComponentBase
<div class="page">
  <div class="sidebar">
    <NavMenu />
  </div>
  <main>
    <div class="top-row px-4">
      <a href="https://docs.microsoft.com/aspnet/" target="_blank">About</a>
    </div>
    <article class="content px-4">
      @Body
    </article>
  </main>
</div>
```

The **@Body** content is provided by the page that satisfies the route. For the first page, that would be `/`. If you look at the files in the **Pages** folder, the **Index.razor** file has a **@page** directive with an argument of `/`. So, by default, that is the page that would be displayed. The **@page** directive is a Razor construct that allows the route to be satisfied without using a controller. The **NavLink** entries in the **Shared/NavMenu.razor** file reference the route using the **href** attribute. That value is looked up in the list of **@page** directives to find a match. If no match is found, then the content in the **<NotFound>** element in the **Main.razor** file is rendered.

Open the **Pages/Counter.razor** page to see how a Razor page works:

```

@page "/counter"
<h1>Counter</h1>
<p role="status">Current count: @currentCount</p>
<button class="btn btn-primary" @onclick="IncrementCount">Click me</button>
@code {
    private int currentCount = 0;
    private void IncrementCount()
    {
        currentCount++;
    }
}

```

After the **@page** directive, the page is made up of HTML mixed with some Razor directives. Then, there's the **@code** directive, which contains the C# code for the page. The HTML **button** element has its click event mapped to the C# **IncrementCount** method using the **@onclick** directive.

LEARN MORE

To learn more about Razor pages, take a look at the official documentation at <https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-7.0>.

Now, let's start creating the project.

Preparing the project

.NET MAUI and Blazor integrate seamlessly – so seamlessly that it is sometimes hard to differentiate what is executing where. That makes it very easy to render data in both XAML and HTML.

Let's get started by preparing the project for the calculator. We will start by removing much of the template code we won't be using. Follow these steps to prepare the template:

1. In Visual Studio, using **Solution Explorer**, delete the **Data** folder.
2. In the **Pages** folder, delete **Index.razor**, **Counter.Razor**, and **FetchData.razor**.
3. In the **Shared** folder, delete **NavMenu.razor**, **NavMenu.razor.css**, and **SurveyPrompt.razor**.
4. Right-click on the **Pages** folder, then select **Add | Razor Component...**, as shown in the following screenshot:

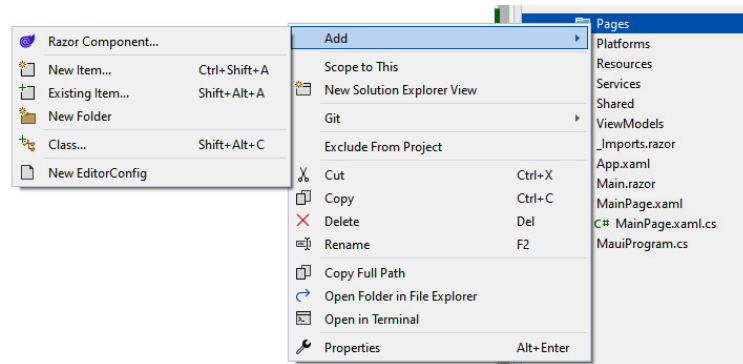


Figure 11.7 – Adding a new Razor component

5. In the **Add New Item** dialog, name the component **Keypad.razor** and click **Add**:

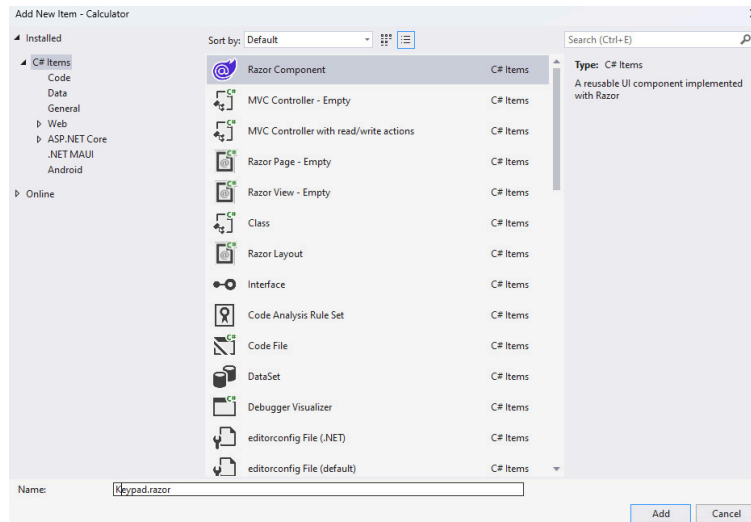


Figure 11.8 – Razor component

6. In the new **Keypad.razor** file, add the following highlighted lines:

```
@page "/"
<h3>Keypad</h3>
<div>Keypad goes here</div>
@code {
}
```

7. Open the **MainLayout.razor** file in the **Shared** folder and remove the following highlighted sections:

```
@inherits LayoutComponentBase
<div class="page">
  <div class="sidebar">
    <NavMenu />
  </div>
  <main>
    <div class="top-row px-4">
      <a href="https://docs.microsoft.com/aspnet/" target="_blank">About</a>
    </div>
    <article class="content px-4">
      @Body
    </article>
  </main>
</div>
```

8. Open **MauiProgram.cs** and remove the highlighted lines of code:

```
using Calculator.Data;
using Microsoft.Extensions.Logging;
namespace Calculator
{
  public static class MauiProgram
  {
    public static MauiApp CreateMauiApp()
    {
      var builder = MauiApp.CreateBuilder();
```

```

        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            });
        builder.Services.AddMauiBlazorWebView();
    #if DEBUG
        builder.Services.AddBlazorWebViewDeveloperTools();
        builder.Logging.AddDebug();
    #endif
        builder.Services.AddSingleton<WeatherForecastService>();
    return builder.Build();
}
}
}

```

9. Now, run the project to see the effects of the changes, as shown in *Figure 11.9*:



Figure 11.9 – Running the blank calculator app on Windows

Now that the project is ready to be an app, let's start with the Keypad view.

Creating the Keypad view

The Keypad view is the basic functionality of the calculator. It has buttons for each digit, 0 through 9, the decimal separator, ., clear, C, clear everything, CE, and, finally, addition, +, subtraction, -, multiplication, x, division, /, = to get the result, and < to erase the last character. The view has three basic components to it – HTML and CSS for styling and the C# code. Each of the following sections will guide you through adding each of these components for the view, starting with the HTML.

Adding the HTML

Razor files and .NET MAUI XAML files do not have a designer for you to visualize your changes. You have to run your app to see the changes you have made, which usually involves building, deploying, and then navigating to the view in your app. .NET has a time-saving feature called Hot Reload. It works by applying the changes you make to your Razor, XAML, CSS, and C# files to the running application without you having to stop and restart the app. Try using **Hot Reload** while you go through the next set of steps. To apply the changes, use the **Hot Reload** button in the **Debug** toolbar. It's easy to find – it's the flames icon:

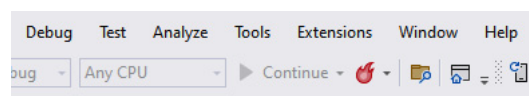


Figure 11.10 – The Hot Reload toolbar button

If at any point you receive a “Rude Edit” dialog like the following, it just means that the change cannot be applied by Hot Reload, so you need to stop, rebuild, and restart debugging:

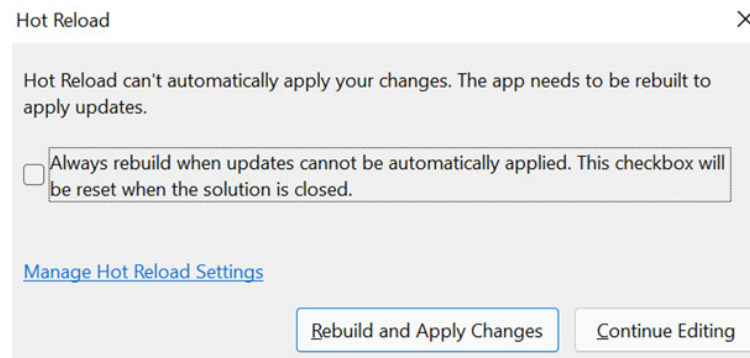


Figure 11.11 – Hot Reload dialog

Hot Reload is constantly getting better, so there will be fewer and fewer of these cases in the future.

To add the HTML that will provide you with the UI to interact with the keypad, follow these steps:

1. Start the app by using the debugger or pressing *F5*.
2. If you want to see your app in Visual Studio, you can open the **XAML Live Preview** pane. To open the **XAML Live Preview** pane, use the Visual Studio menu and select **Debug | Windows | XAML Live Preview**. I usually pin it open so that it is always available.
3. Open the **Keypad.razor** file in the **Pages** folder.
4. Remove the highlighted content shown in the following code block:

```
@page "/"
<h3>Keypad</h3>
<div>Keypad goes here</div>
@code {
}
```

5. Add the following HTML after **@page** but before the **@code** directive:

```
<div class="keypad">
  <div class="keypad-body">
    <div class="keypad-screen">
      <div class="keypad-typed"></div>
    </div>
    <div class="keypad-row">
      <div class="keypad-button wide command">C</div>
      <div class="keypad-button command">CE</div>
      <div class="keypad-button operator">/</div>
    </div>
    <div class="keypad-row">
      <div class="keypad-button">7</div>
      <div class="keypad-button">8</div>
      <div class="keypad-button">9</div>
      <div class="keypad-button operator">X</div>
    </div>
    <div class="keypad-row">
      <div class="keypad-button">4</div>
```

```

<div class="keypad-button">5</div>
<div class="keypad-button">6</div>
<div class="keypad-button operator">-</div>
</div>
<div class="keypad-row">
<div class="keypad-button">1</div>
<div class="keypad-button">2</div>
<div class="keypad-button">3</div>
<div class="keypad-button operator">+</div>
</div>
<div class="keypad-row">
<div class="keypad-button">.</div>
<div class="keypad-button">0</div>
<div class="keypad-button">&lt;</div>
<div class="keypad-button operator">=</div>
</div>
</div>
</div>

```

Don't forget to press the **Hot Reload** button to see your changes! The keypad is implemented using **div** and not **table** since it is easier to style **div** elements than **table** elements. The keypad is arranged in rows of four buttons, except for the top two rows. The top row is the display where the expression or result is shown. The second row only has three buttons. The class names have already been added to the HTML elements, but since they don't exist yet, they don't alter the appearance.

The keypad doesn't quite look like a keypad yet, but it will once we add some style to it.

Adding style to the HTML

CSS has been around for a very long time and is the best way to make your HTML look its best. Follow these steps to add the styles we used in the previous section:

1. Make sure your app is still running and use **Solution Explorer** to add a new CSS stylesheet file named **Keypad.razor.css** in the **Pages** folder. Click the **Show All Templates** button if Visual Studio doesn't display them by default:

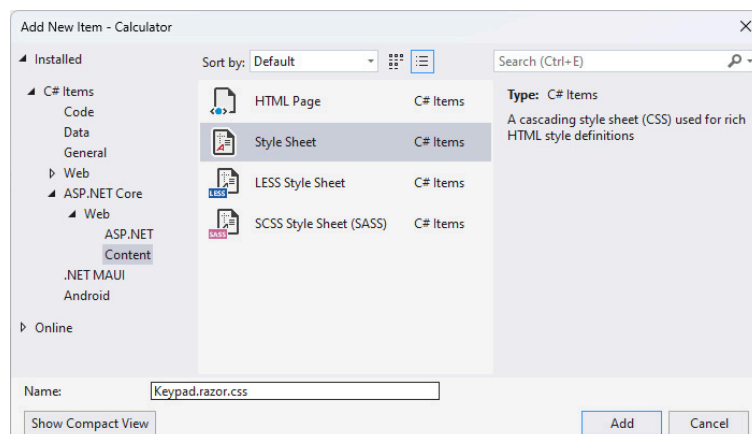


Figure 11.12 – Adding a new CSS file

If you look in **Solution Explorer** under the **Pages** folder, you will notice that your new file is now under the **Keypad.razor** file:

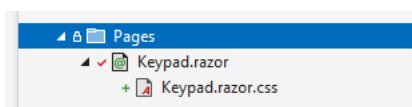


Figure 11.13 – A Razor page with an isolated CSS file

Visual Studio automatically recognized that you wanted to add an **isolated CSS** file and made the association. Isolated CSS files are a feature of ASP.NET that allows you to write CSS that only affects the Razor file it's associated with. None of the styles you add to this file can be used outside of the **Keypad.razor** file.

2. Add the **keypad** style to the **Keypad.razor.css** file:

```
.keypad {
    width: 300px;
    margin: auto;
    margin-top: -1.1em;
}
```

This style will set the width of the element to **300px**, with automatic margins, except for the top, which is at **-1.1em**. **-1.1em** moves the top edge of the keypad right up against the top of the web view control.

3. Now, add the **keypad-body** style using the following code:

```
.keypad-body {
    border: solid 1px #3A4655;
}
```

This style just gives the entire element a border one pixel wide in a dark gray color.

4. We will save the **keypad-screen** and **keypad-typed** styles for last, so add the **keypad-row** style shown in the following code:

```
.keypad-row {
    width: 100%;
    background: #3C4857;
}
```

This style sets the element width to 100% of its parent, which would be **keypad-body**, and sets the background to a pleasing dark gray color.

5. The next style to add is **keypad-button**. Use the following code to add the style:

```
.keypad-button {
    width: 25%;
    background: #425062;
    color: #fff;
    padding: 20px;
    display: inline-block;
    font-size: 25px;
    text-align: center;
    vertical-align: middle;
    margin-right: -4px;
}
```

```
border-right: solid 2px #3C4857;
border-bottom: solid 2px #3C4857;
transition: all 0.2s ease-in-out;
}
```

This style is the base for all the buttons on the keypad, hence it has the most attributes. Elements that have this style applied have a border on the right and bottom that is 2 pixels wide and uses the same color as the background of the row. The **background** attribute of the button is a bit darker than the border color, which provides a bit of depth. The text is aligned in the center both vertically and horizontally and uses a 25-pixel font size. The width is set to 25% since there are four buttons in a row, mostly. The **transition** attribute imposes a 200-millisecond transition using **ease-in-out**, which accelerates from the start to the middle and then decelerates from the middle to the end. **transition** is applied to all attributes, so any time an attribute of this style changes, it will slowly change from the starting value to the end value.

6. If the button is an action button, such as an operator, then an additional style is applied, called **operator**. The definition of this style is slightly different from the rest of the styles that have been created so far. Instead of just being named **operator**, this style is named **keypad-button.operator**. In CSS, **.** is a selector; it is used to locate which attributes to apply. In this case, we want all elements that have both the **keypad-button** class and the **operator** class applied. To add the **keypad-button.operator** class, use the following code:

```
.keypad-button.operator {
    color: #AEB3BA;
    background: #404D5E;
}
```

These buttons will appear with a slightly darker background with slightly less white text.

7. The clear (C) and clear everything (CE) buttons also have their own class, as follows:

```
.keypad-button.command {
    color: #D95D4E;
    background: #404D5E;
}
```

These buttons will appear with slightly darker backgrounds with red text.

8. Now, for the desktop, we can add hover highlighting by using the **:hover** pseudo selector. Add the hover styles using the following code:

```
.keypad-button:hover {
    background: #E0B612;
}
.keypad-button.command:hover,
.keypad-button.operator:hover {
    background: #E0B612;
    color: #fff;
}
```

The background is altered to an orange color. The change isn't immediate due to the **transition** attribute on the **keypad-button** style, so it will transition from dark gray to orange in two-tenths of a second.

9. The last button-related style is wide, or **keypad-button.wide**. This style makes the button twice as wide as a normal button. To add this style, use the following code:

```
.keypad-button.wide {  
    width: 50%;  
}
```

10. The final two styles, **keypad-screen** and **keypad-typed**, are for displaying the expression and the result. Use the following code to add the remaining two styles:

```
.keypad-screen {  
    background: #3A4655;  
    width: 100%;  
    height: 75px;  
    padding: 20px;  
}  
.keypad-typed {  
    font-size: 45px;  
    text-align: right;  
    color: #fff;  
}
```

Now, the keypad looks like a real calculator keypad; refer to *Figure 11.14* for an example. Were you able to keep adding styles and seeing the changes in your app without having to restart? Remember to click the **Hot Reload** button or set the **Hot Reload on File Save** option under the **Hot Reload** button menu; Visual Studio will attempt to apply the changes as soon as you save the file. Next, we will add the code to enable the buttons to work:

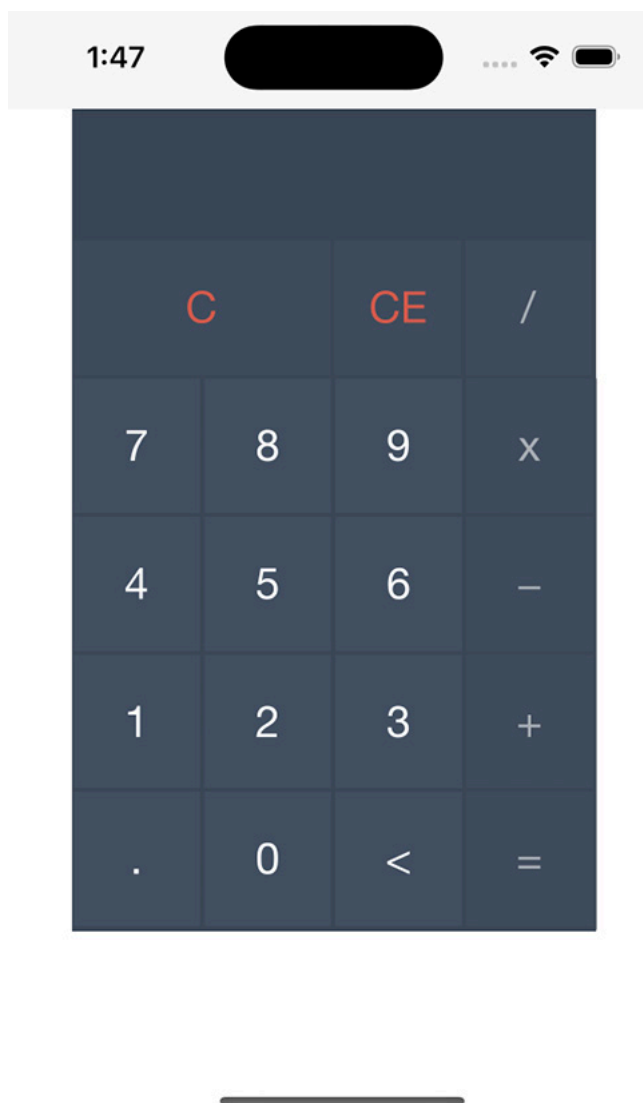


Figure 11.14 – HTML keypad with style

Wiring up the controls

In most web pages, you would use **JavaScript** to write the code to interact with the page. In Blazor, you use C#. To add the C# functions to the **Keypad.razor** file, follow these steps:

1. Within the **@code** directive block, add the following:

```
string inputDisplay = string.Empty;  
bool clearInputBeforeAppend = false;
```

This declares a field named **inputDisplay** as a **string** element and initializes it to the empty string. It also declares a **bool** element and initializes it to **false**. **clearInputBeforeAppend** is a flag to keep **inputDisplay** clean. After a result is displayed, when the user taps a button, **inputDisplay** should be cleared before the character is added to the screen.

2. Update the element with the **keypad-typed** class, as in the following code:


```
<div class="keypad-typed">@inputDisplay</div>
```

This will render the contents of the **inputDisplay** variable into the **div** element. Note the use of **@** to reference the C# field.

3. To help with validating the input, add the following:

```
readonly char[] symbols = { '/', 'X', '+', '-', '.' };
```

4. When any of the numbers (0 through 9) or the operator buttons are tapped, **inputDisplay** will be updated by appending a character to the display. Add the **AppendInput** method using the following code:

```
void AppendInput(string inputValue)
{
    double numValue;
    if (clearInputBeforeAppend)
    {
        inputDisplay = string.Empty;
    }
    if (string.IsNullOrEmpty(inputDisplay) && inputValue.IndexOfAny(symbols) != -1)
    {
        return;
    }
    if (!double.TryParse(inputValue, out numValue) && !string.IsNullOrEmpty(inputDisplay) && $"
    {
        return;
    }
    if (inputDisplay.Trim() == "0" && inputValue == "0")
    {
        return;
    }
    clearInputBeforeAppend = false;
    inputDisplay += inputValue;
}
```

Let's review the code. The first check is to see whether **inputDisplay** needs to be cleared; if so, it's cleared. Then, a check for operators is made. The next check is more complex as it doesn't allow multiple operators in a row. This check uses the **Range** syntax of **^1** to indicate the last character. String interpolation is used to convert the last character back into a string, so **IndexOfAny** can be used to find that character in the array of symbols. Multiple leading 0s are checked and rejected if found. If all the checks pass, the input is appended to **inputDisplay**, and the **clearInputBeforeAppend** flag is reset to **false**.

5. When the user presses the < button, it will remove the last character from the input. Add the **Undo** method using the following code:

```
void Undo()
{
    if (!clearInputBeforeAppend && inputDisplay.Length > 0)
    {
        inputDisplay = inputDisplay[0..^1];
        return;
    }
}
```

This method uses the **Range** syntax again, and the capability of strings being able to be indexed as arrays. It uses the array syntax to get the elements from index 0 to the next to last index and returns it.

6. When the user presses the **C** button, it will clear the input display. Add the **ClearInput** method using the following code:

```
void ClearInput()
{
    inputDisplay = string.Empty;
}
```

7. When the user presses the **CE** button, it will clear everything. For now, that is just the input display. Use the following code to add the **ClearAll** method:

```
void ClearAll()
{
    ClearInput();
}
```

8. Finally, the **=** button will take the expression that was entered and evaluate the result. Use the following code to add the **EvaluateExpression** method:

```
void EvaluateExpression()
{
    var expression = inputDisplay;
    clearInputBeforeAppend = true;
}
```

This method doesn't evaluate the expression that's entered yet; that will happen in the *Creating the Compute service* section.

The next step in the **Keypad.razor** file is to hook the methods that were just defined so that they are called when the user taps or clicks on that element. Like in standard HTML, events are hooked up via attributes on the element that reference code, either inline or methods. The attributes in Razor pages use a page directive with the event name. For example, the DOM event for handling taps or clicks is **click**, so the Razor event name would be **@onclick**. That attribute is then assigned to a delegate, which could be any method. The full attribute might look like **@onclick="DoSomething"**, where **DoSomething** is a C# method defined in the page's **@code** directive.

The **AppendInput** method accepts a single string argument, so the delegate cannot be just **AppendInput** – it has to be wrapped in an expression so that the parameter can be passed along. Expressions in Razor pages are contained in the **@(...)** directive. All calls to **AppendInput** from an event directive would look like **@onclick="@ (AppendInput ("0"))"**.

Use the highlighted lines in the following code to update the HTML in the **Keypad.razor** file:

```
<div class="keypad">
```

```

<div class="keypad-body">
  <div class="keypad-screen">
    <div class="keypad-typed">@inputDisplay</div>
  </div>
  <div class="keypad-row">
    <div class="keypad-button wide command" @onclick="ClearInput">C</div>
    <div class="keypad-button command" @onclick="ClearAll">CE</div>
    <div class="keypad-button operator" @onclick="@(()=>AppendInput("/"))">/</div>
  </div>
  <div class="keypad-row">
    <div class="keypad-button" @onclick="@(()=>AppendInput("7"))">7</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("8"))">8</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("9"))">9</div>
    <div class="keypad-button operator" @onclick="@(()=>AppendInput("X"))">X</div>
  </div>
  <div class="keypad-row">
    <div class="keypad-button" @onclick="@(()=>AppendInput("4"))">4</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("5"))">5</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("6"))">6</div>
    <div class="keypad-button operator" @onclick="@(()=>AppendInput("-"))">-</div>
  </div>
  <div class="keypad-row">
    <div class="keypad-button" @onclick="@(()=>AppendInput("1"))">1</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("2"))">2</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("3"))">3</div>
    <div class="keypad-button operator" @onclick="@(()=>AppendInput("+"))">+</div>
  </div>
  <div class="keypad-row">
    <div class="keypad-button" @onclick="@(()=>AppendInput("."))">.</div>
    <div class="keypad-button" @onclick="@(()=>AppendInput("0"))">0</div>
    <div class="keypad-button" @onclick="Undo">&lt;</div>
    <div class="keypad-button operator" @onclick="EvaluateExpression">=</div>
  </div>
</div>

```

MORE ABOUT RAZOR EVENT HANDLING

To learn more about Razor event handling, visit

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/event-handling?view=aspnetcore-7.0>.

The keypad is nearly complete. At this point, you should be able to enter a full expression to evaluate and then clear the display. In the next section, you will create a service to evaluate the expression.

Creating the Compute service

The **Compute** service evaluates the expression and returns the result. To illustrate how .NET MAUI and the Blazor application can interact with one another, this service will be injected from the .NET MAUI dependency injection container into the Blazor page. To implement the **Compute** service, follow these steps:

1. Create a new folder in the root of the project named **Services**.
2. In the **Services** folder, add a new C# class file named **Compute**.
3. Modify the **Compute.cs** file so that it matches the following code:

```
namespace Calculator.Services;
internal class Compute
{
    public string Evaluate(string expression)
    {
        System.Data.DataTable dataTable = new System.Data.DataTable();
        var finalResult = dataTable.Compute(expression, string.Empty);
        return finalResult.ToString();
    }
}
```

This code might be shorter than you expected. Instead of writing a lot of code to parse the expression and build an expression tree to evaluate, there is already a built-in way of evaluating simple expressions: **DataTable**. The **DataTable.Compute** method can evaluate all the expressions that can be built from the calculator.

4. Open the **MauiProgram.cs** file and add the following highlighted line of code to register the class with dependency injection:

```
#if DEBUG
    builder.Services.AddBlazorWebViewDeveloperTools();
    builder.Logging.AddDebug();
#endif
    builder.Services.AddSingleton<Compute>();
    return builder.Build();
```

5. To allow the **Keypad.razor** page to use the **Compute** type, it needs to be declared with Razor. Open the **_Imports.razor** file and add the following highlighted line of code to the end of the file:

```
@using Calculator.Services
```

Without this line, you could still use the type, but you would have to fully qualify it as **Calculator.Services.Compute**. This is the Razor equivalent of **global using** directives in C# files.

6. Now, open the **Keypad.razor** file and add the following line of code after the **@page** directive:

```
@inject Compute compute
```

The **@inject** directive will use the .NET MAUI dependency injection container to resolve the type provided as the first parameter and assign it to the variable defined in the second parameter.

7. In the **EvaluateExpression** method, the **compute** variable can be used to evaluate the expression contained in **inputDisplay**, as shown in the following highlighted code:

```
void EvaluateExpression()
{
    var expression = inputDisplay;
    var result = compute.Evaluate(inputDisplay.Replace('X', '*'));
    inputDisplay = result;
    clearInputBeforeAppend = true;
}
```

Here, the **Evaluate** method is called with **inputDisplay** as the parameter. **inputDisplay** is first modified to replace all the **X** values in the string with ***** as that is what the **DataTable** expects for multiplication. The result is then assigned to **inputDisplay**.

At this point, the calculator app can accept input of a combination of numbers and operators and evaluate the result, displaying it to the user. The user can also clear out the display. In the next section, we will explore more interoperability by giving the calculator memory.

Adding memory functions

Most calculators can store previous calculations. In this section, you will add a list of previous calculations to the calculator app and recall previous calculations to the **inputDisplay** parameter of the keypad.

The code will use .NET MAUI controls to render right next to **WebViewControl**. A new class, **Calculations**, will be used to manage the listing.

To add the memory functions to the calculator app, follow these steps:

1. Create a new folder named **ViewModels**.
2. Add a new class named **Calculations** in the **ViewModels** folder and modify the file so that it matches the following code:

```
using System.Collections.ObjectModel;
namespace Calculator.ViewModels;
public class Calculations : ObservableCollection<Calculation>
{
}
public class Calculation : Tuple<string, string>
{
    public Calculation(string expression, string result) : base(expression, result) { }
    public string Expression => this.Item1;
    public string Result => this.Item2;
}
```

This code adds two classes – **Calculations**, which is just shorthand for **ObservableCollection<Calculation>**, and **Calculation**, which is **Tuple<string, string>** and defines two properties to reference **Item1** as **Expression** and **Item2** as **Result**.

3. Add a reference to the **CommunityToolkit.Mvvm** NuGet package.
4. Add a new class named **MainPageViewModel** to the **ViewModels** folder and modify the file, as shown in the following code:

```
using CommunityToolkit.Mvvm.Input;
using CommunityToolkit.Mvvm.Messaging;
namespace Calculator.ViewModels;
public partial class MainPageViewModel
{
    IMessenger messenger;
    public MainPageViewModel(Calculations results, IMessenger messenger)
    {
        Results = results;
    }
}
```

```

        this.messenger = messenger;
    }
    public Calculations Results { get; init; }
    [RelayCommand]
    public void Recall(Calculation sender)
    {
        messenger.Send(sender);
    }
}

```

The **MainViewModel** class uses two features of **CommunityToolkit**: **RelayCommand** and **IMessenger**. As in the other chapters, **RelayCommand** is used to bind methods to XAML actions. **IMessenger** is an interface for sending messages between parts of your application. It is helpful when you don't want to take a hard dependency between two classes, especially if it creates a circular reference. **CommunityToolkit** provides a default implementation of **IMessenger** called **WeakReferenceMessenger**.

5. Open **MauiProgram.cs** and add the following **using** declarations to the top of the file:

```

using Calculator.ViewModels;
using CommunityToolkit.Mvvm.Messaging;

```

6. In the **CreateMauiApp** method make the following highlighted changes:

```

builder.Services.AddSingleton<Compute>();
builder.Services.AddSingleton<Calculations>();
builder.Services.AddSingleton<MainPage>();
builder.Services.AddSingleton<MainPageViewModel>();
builder.Services.AddSingleton<IMessenger>(WeakReferenceMessenger.Default);
return builder.Build();

```

This adds **MainPage**, **MainPageViewModel**, and the default instance of **WeakReferenceMessenger** to the dependency injection container. The next few steps will enable **MainPage** to be initialized via dependency injection.

7. Open **MainView.xaml.cs** and make the following highlighted changes:

```

using Calculator.ViewModels;
namespace Calculator;
public partial class MainPage : ContentPage
{
    public MainPage(MainPageViewModel vm)
    {
        InitializeComponent();
        BindingContext = vm;
    }
}

```

As in other chapters, the constructor for the view is updated to accept the view model as a parameter, and it is assigned as **BindingContext**.

8. Open **App.xaml.cs**, modify the constructor, and add the **OnHandlerChanging** event handler, as shown in the following code:

```

public App()
{

```

```

        InitializeComponent();
    }
    protected override void OnHandlerChanging(HandlerChangingEventArgs args)
    {
        base.OnHandlerChanging(args);
        MainPage = args.NewHandler.MauiContext.Services.GetService<MainPage>();
    }

```

Since a .NET MAUI Blazor app does not use **Shell** by default, the views cannot be initialized via dependency injection the same way as is done with **Shell**. Instead, the instance of **MainPage** is obtained after **Handler** has been set. The **OnHandlerChanging** event is used to get the reference to the new **Handler**, which then provides **MauiContext** to the dependency injection container.

9. Open **_Imports.razor** and add the following lines to the end of the file:

```

@using Calculator.ViewModels
@using CommunityToolkit.Mvvm.Messaging

```

10. Open **Keypad.razor** and add the following highlighted lines:

```

@Inject Compute compute
@Inject Calculations calculations
@Inject IMessenger messenger
<div class="keypad">

```

This will inject the **Calculations** instance as **calculations** and **WeakReference Messenger** as **messenger** from the .NET MAUI dependency injection container.

11. Modify the **ClearAll** and **EvaluateExpression** methods and add an **OnAfterRenderAsync** method, as shown in the following code:

```

void ClearAll()
{
    ClearInput();
    calculations.Clear();
}
void EvaluateExpression()
{
    var expression = inputDisplay;
    var result = compute.Evaluate(inputDisplay.Replace('X', '*'));
    calculations.Add(new(expression, result));
    inputDisplay = result;
    clearInputBeforeAppend = true;
}
protected override Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        messenger.Register<Calculation>(this, (sender, er) =>
        {
            inputDisplay = er.Expression;
            clearInputBeforeAppend = true;
            StateHasChanged();
        });
    }
}

```

```

        return base.OnAfterRenderAsync(firstRender);
    }

```

ClearAll will just clear the collection, and **EvaluateExpression** will add the new **Calculation** to the collection. **OnAfterRenderAsync** is used to **register** this class to receive messages for any **Calculation** objects. When a message is received, **inputDisplay** is set to the **Expression** value of **Calculation**, and **StateHasChanged** is called to force the UI to refresh with the updated value.

12. Open the **Shared/MainLayout.razor.css** file and add the following line to the **page** class:

```
background-color: black;
```

This is simply for aesthetics and makes the areas surrounding the calculator black.

13. Open the **MainPage.xaml** file and modify it so that it matches the following code:

```

<ContentPage
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:Calculator"
    xmlns:viewModels="clr-namespace:Calculator.ViewModels"
    x:Class="Calculator.MainPage"
    x:DataType="viewModels:MainPageViewModel">
    <Grid BackgroundColor="Black">
        <Grid.RowDefinitions>
            <RowDefinition Height="1*" />
            <RowDefinition Height="1*" />
        </Grid.RowDefinitions>
        <ScrollView Grid.Row="0" BackgroundColor="Bisque" WidthRequest="400" VerticalScrollBarVis
            <CollectionView ItemsSource="{Binding Results}" ItemsUpdatingScrollMode="KeepLastItemIn
                <CollectionView.ItemTemplate>
                    <DataTemplate x:DataType="viewModels:ExpressionResult">
                        <SwipeView>
                            <SwipeView.LeftItems>
                                <SwipeItems Mode="Execute">
                                    <SwipeItem
Text="Recall"                                BackgroundColor="LightPink"
                                    />
                                </SwipeItems>
                            </SwipeView.LeftItems>
                        <VerticalStackLayout>
                            <HorizontalStackLayout Padding="10" HorizontalOptions="EndAndExpand">
                                <Label Text="{Binding Expression}" FontSize="Large" TextColor="Black" Hor
                                <Label Text="=" TextColor="Blue" FontSize="Large" HorizontalTextAlignment
                                <Label Text="{Binding Result}" FontSize="Large" TextColor="Black" Horizon
                            </HorizontalStackLayout>
                            <Line Stroke="LightSlateGray" X2="400" />
                            <Line Stroke="Black" X2="400" />
                        </VerticalStackLayout>
                    </SwipeView>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </ScrollView>
    <BlazorWebView Grid.Row="1" x:Name="blazorWebView" HostPage="wwwroot/index.html" HeightRe
        <BlazorWebView.RootComponents>

```



```

        <RootComponent Selector="#app" ComponentType="{x:Type local:Main}" />
    </BlazorWebView.RootComponents>
</BlazorWebView>
</Grid>
</ContentPage>

```

A **Grid** element with two rows defined is used to contain the new display for previous calculations and the original **BlazorWebView** control. The calculations are rendered using **ScrollView**, which contains **CollectionView**. The **CollectionView.ItemTemplate** property contains **DataTemplate** for each **Calculation**. The **SwipeView** control allows the user to swipe in either the up, down, left, or right direction to expose additional commands. Each direction has an element to define those actions. For **Calculation** items, when the user swipes right, it exposes a **Recall** item, which is bound to the **Recall** command of **MainPageViewModel**. The display for **Calculation** uses a combination of horizontal and vertical **StackLayout** controls to stack **Expression** on top of **Result** with **=**, all aligned to the left.

That completes the main functionality of our calculator app. The next section deals with some aesthetics of the main window when running on a desktop such as Windows or macOS.

Resizing the main window

The calculator app is defined to be a fixed size. Most of the projects in this book have allowed the controls to grow or shrink with changes to the window size. For our calculator app, the main window should be fixed so that it appears in the best way possible. To fix the window size when the app launches, open the **App.xaml.cs** file and add the following method to the **App** class:

```

protected override Window CreateWindow(IActivationState activationState)
{
    var window = base.CreateWindow(activationState);
    if (OperatingSystem.IsWindows() || OperatingSystem.IsMacCatalyst())
    {
        window.Created += Window_Created;
    }
    return window;
}
private async void Window_Created(object sender, EventArgs e)
{
    const int defaultWidth = 450;
    const int defaultHeight = 800;
    var window = (Window)sender;
    window.Width = defaultWidth;
    window.Height = defaultHeight;
    window.X = -defaultWidth;
    window.Y = -defaultHeight;
    await window.Dispatcher.DispatchAsync(() => { });
    var displayInfo = DeviceDisplay.Current.MainDisplayInfo;
    window.X = (displayInfo.Width / displayInfo.Density - window.Width) / 2;
    window.Y = (displayInfo.Height / displayInfo.Density - window.Height) / 2;
}

```

```
window.Created -= Window_Created;  
}
```

The **CreateWindow** method is overridden so that the **Window.Created** event can have a custom handler attached to it if the app is running on Windows or macOS. The work of resizing the window is done in the **Window_Created** method. It uses the **defaultHeight** and **defaultWidth** constants to set the new windows' height, width, and location on the screen. The method then waits for all threads to have finished before altering the **X** and **Y** properties of the window again, but this time accounting for the screen pixel density. Finally, it detaches the event handler

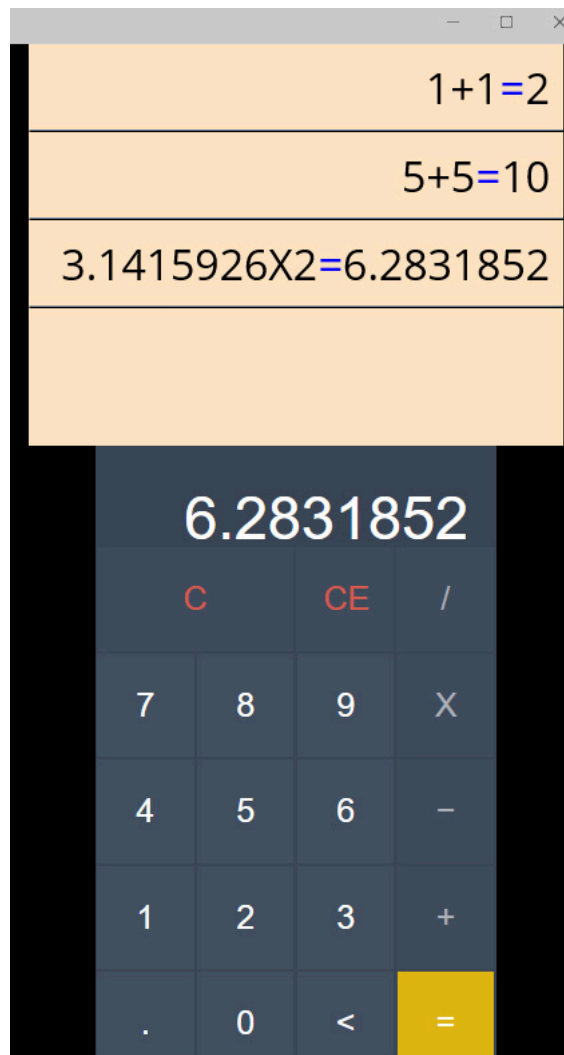


Figure 11.15 – The completed calculator project

Summary

Excellent work! In this chapter, you completed a project that used the .NET MAUI Blazor template. You created a UI using HTML, updated it with C# code, and then implemented a service that was managed by .NET MAUI and injected it into a Razor page. Then, you used **CollectionView** to display a list of the previous calculations. Within **CollectionView ItemTemplate**, the **SwipeView** control was used to recall a previous calculation to the keypad for additional editing and reevaluation.

To extend this project even further, consider the following:

- Add an additional swipe action to delete a calculation from the collection
- Add an additional keypad layout for scientific calculations

In the next – and final – chapter, you will discover the world of AI as you build an object recognition app.